





**Institute of Technology of Cambodia**

Department of Applied Mathematics and Statistics

Submission date: 29 October 2026

# Report: Introduction to OOP

| Name | Student ID | Score |
|---|---|---|
| VIN SOVICHHAY | e20241261 | ..... |

**Lecturers:** Mr. LONG Pakrigna (Course)
Mr. VANN Visal (TP)

**Academic Year 2026–2027**

# Contents


**1 Python Basic Data Types** **1**

1.1 Primitive/Built-in Data Types . . . . . 1

1.2 Collection and Reference Types . . . . . 2

1.3 Literals in Python . . . . . 2

1.4 Practical Example: Student Grade Management . . . . . 3

**2 Python Variables and Attributes** **4**

2.1 Local Variables . . . . . 4

2.2 Instance Attributes . . . . . 4

2.3 Class Attributes: . . . . . 5

2.4 Practical Example: Student Information System . . . . . 6

**3 Python Modifiers and Access Control — Part 1** **6**

3.1 Public, Non-Public, and Private Naming Conventions . . . . . 7

3.2 Access Control by Convention and Name Mangling . . . . . 7

3.3 The `@staticmethod` Decorator . . . . . 8

3.4 The `@classmethod` Decorator . . . . . 9

3.5 Practical Example: Bank Account Management . . . . . 9

**4 Python Modifiers and OOP Features — Part 2** **10**

4.1 Abstract Classes and Methods . . . . . 10

4.2 Access Control and Encapsulation . . . . . 11

4.3 Other OOP Features . . . . . 12

4.3.1 The `@property` Decorator . . . . . 12

4.3.2 The `@final` Decorator and `typing.final` . . . . . 13

4.4 Practical Example: Shape Area Calculator . . . . . 13

**5 Python Operators — Part 1** **14**

5.1 Arithmetic Operators . . . . . 14

5.2 Comparison Operators . . . . . 15

5.3 Bitwise Operators . . . . . 16

5.4 Logical Operators . . . . . 17

5.5 Practical Example: Student Grades and Eligibility . . . . . 17

**6 Python Operators — Part 2** **18**

6.1 Assignment Operators . . . . . 18

6.2 Special Operators . . . . . 19
6.2.1 Identity Operators . . . . . 20
6.2.2 Membership Operators . . . . . 20
6.2.3 Conditional Expressions . . . . . 21
6.3 Operator Precedence and Associativity . . . . . 21
6.4 Practical Example: Student Record Management . . . . . 22

**7 Python Decision-Making Statements — Part 1 23**
7.1 The `if` Statement . . . . . 23
7.2 The `if-else` Statement . . . . . 24
7.3 The `if-elif-else` Statement . . . . . 24
7.4 Practical Example: Student Pass/Fail Status and Letter Grades . . . . . 25

**8 Python Decision-Making Statements — Part 2 26**
8.1 Nested `if` Statements . . . . . 26
8.2 Multiple-Branch Selection Using `match-case` . . . . . 26
8.3 Conditional Expressions . . . . . 27
8.4 Practical Example: Menu-Driven Student Management System . . . . . 28

**9 Python Loops and Loop Control 30**
9.1 The `while` Loop . . . . . 31
9.2 The `for` Loop . . . . . 31
9.3 Repetition with `for` and `while` . . . . . 31
9.4 Loop Control Statements: `break`, `continue`, and `pass` . . . . . 32
9.5 The `break` Statement . . . . . 33
9.6 Practical Example: Student Objects, Averages, and Searching . . . . . 34
9.7 Important Differences Between Java and Python . . . . . 35

**10 Conclusion 36**

**11 References 37**

# 1 Python Basic Data Types

In Python, data types define the kind of value that a variable can store and the operations that can be performed on that value. Understanding data types is essential in programming because they help organize information and ensure that values are used correctly. Python provides several built-in data types, including numeric types, strings, Boolean values, and collection types.

## 1.1 Primitive/Built-in Data Types

Python provides several fundamental built-in data types for representing different kinds of values.

- **Integer (`int`):** Represents whole numbers without a decimal point, such as `10`, `-5`, and `0`.
- **Float (`float`):** Represents numbers with decimal points, such as `3.14` and `-2.5`.
- **Boolean (`bool`):** Represents logical values, either `True` or `False`.
- **String (`str`):** Represents text enclosed in single or double quotation marks.
- **Complex (`complex`):** Represents complex numbers containing real and imaginary parts, such as `2 + 3j`.
- **NoneType:** Represents the absence of a value using the special value `None`.

The built-in `type()` function can be used to determine the data type of a value or variable.

**Python Code:**

```
age = 20
average = 85.5
is_enrolled = True
name = "Dara"
result = 2 + 3j
address = None

print(type(age))
print(type(average))
print(type(is_enrolled))
print(type(name))
print(type(result))
print(type(address))
```

**Output:**

```
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'str'>
<class 'complex'>
<class 'NoneType'>
```

## 1.2 Collection and Reference Types

Collection types allow multiple values to be stored in a single object. Python provides four commonly used collection types: lists, tuples, sets, and dictionaries.

- **List (`list`):** An ordered and mutable collection that allows duplicate values.
- **Tuple (`tuple`):** An ordered collection that cannot be modified after creation.
- **Set (`set`):** A collection of unique elements that does not maintain a guaranteed positional order.
- **Dictionary (`dict`):** A mutable collection that stores values as key-value pairs.

In Python, variables refer to objects rather than directly containing the objects themselves. Multiple variables can refer to the same object. When that object is mutable, changes made through one reference may also be observed through another reference.

**Python Code:**

```
subjects = ["Python", "Mathematics", "Database"]
student = {
    "name": "Dara",
    "age": 20,
    "subjects": subjects
}

print(student["name"])
print(student["subjects"])
```

**Output:**

```
Dara
['Python', 'Mathematics', 'Database']
```

## 1.3 Literals in Python

Literals are fixed values written directly in source code. They are used to assign values to variables, perform calculations, and represent information.

Common categories of Python literals include:

- **Numeric literals:** Integer, floating-point, and complex values, such as `10`, `3.14`, and `2 + 3j`.
- **String literals:** Text values enclosed in quotation marks, such as `"Hello"`.
- **Boolean literals:** The logical values `True` and `False`.
- **Special literal:** The value `None`, which represents the absence of a value.
- **Collection literals:** Values used to create lists, tuples, dictionaries, and sets.

Python also supports different representations of integer literals, including binary with the prefix `0b`, octal with `0o`, and hexadecimal with `0x`.

**Python Code:**

```
decimal_number = 10
binary_number = 0b1010
octal_number = 0o12
hex_number = 0xA

print(decimal_number)
print(binary_number)
print(octal_number)
print(hex_number)
```

**Output:**

```
10
10
10
10
```

All four variables represent the same integer value, which is `10` in decimal notation.

### 1.4 Practical Example: Student Grade Management

A student grade management system uses different Python data types to store and process student information. The following example demonstrates how these data types can be combined to represent a student's academic record.

**Python Code:**

```
student = {
    "name": "Dara",
    "age": 20,
    "scores": [85, 90, 78],
    "average": 84.33,
    "passed": True,
    "remarks": None
}

print("Student:", student["name"])
print("Age:", student["age"])
print("Scores:", student["scores"])
print("Average:", student["average"])
print("Passed:", student["passed"])
print("Remarks:", student["remarks"])
```

**Output:**

```
Student: Dara
Age: 20
Scores: [85, 90, 78]
Average: 84.33
Passed: True
Remarks: None
```

This example demonstrates how different data types serve different purposes. The string stores the student's name, the integer represents age, the list stores examination scores, the floating-point number represents the average score, the Boolean indicates whether the student passed, and `None` represents a remark that has not yet been assigned.

# 2 Python Variables and Attributes

Variables and attributes are used to store and access information in Python. In object-oriented programming (OOP), understanding the difference between local variables, instance attributes, and class attributes is important because each has a different scope and purpose.

## 2.1 Local Variables

Local variables are variables created inside a function or method. They can generally be accessed only within the function or method where they are defined. They are commonly used to store temporary values during program execution.

**Python Code:**

```
class Student:
    def display_grade(self):
        grade = 90
        print("Grade:", grade)

student = Student()
student.display_grade()
```

**Output:**

```
Grade: 90
```

In this example, `grade` is a local variable defined inside the `display_grade()` method. It cannot be accessed directly outside that method.

## 2.2 Instance Attributes

Instance attributes belong to individual objects created from a class. They are commonly defined using `self` inside the `__init__()` method. Each instance can have its own values for these attributes.

**Python Code:**

```python
class Student:
    def __init__(self, name):
        self.name = name

student1 = Student("Dara")
student2 = Student("Sokha")

print(student1.name)
print(student2.name)
```

**Output:**

```
Dara
Sokha
```

Here, `name` is an instance attribute. The two student objects have different names because each object stores its own value.

### 2.3 Class Attributes:

Class attributes are defined directly inside a class, outside its methods. They are shared by instances that access the attribute through the class, unless an instance overrides it with an attribute of the same name. They are useful for information common to all objects.

**Python Code:**

```python
class Student:
    school = "Institute of Technology of Cambodia"

    def __init__(self, name):
        self.name = name

student1 = Student("Dara")
student2 = Student("Sokha")

print(student1.name, "-", student1.school)
print(student2.name, "-", student2.school)
print(Student.school)
```

**Output:**

```
Dara - Institute of Technology of Cambodia
Sokha - Institute of Technology of Cambodia
Institute of Technology of Cambodia
```

In this example, `name` is an instance attribute because each student has a different name. In contrast, `school` is a class attribute that provides the same school name to both students.

### 2.4 Practical Example: Student Information System

The following example combines local variables, instance attributes, and class attributes in a simple student information system.

**Python Code:**

```python
class Student:
    school = "Institute of Technology of Cambodia"

    def __init__(self, name, score):
        self.name = name
        self.score = score

    def display_info(self):
        status = "Passed" if self.score >= 50 else "Failed"

        print("Student:", self.name)
        print("Score:", self.score)
        print("Status:", status)
        print("School:", Student.school)
        print("-" * 30)

student1 = Student("Dara", 85)
student2 = Student("Sokha", 45)

student1.display_info()
student2.display_info()
```

**Output:**

```
Student: Dara
Score: 85
Status: Passed
School: Institute of Technology of Cambodia
------------------------------
Student: Sokha
Score: 45
Status: Failed
School: Institute of Technology of Cambodia
------------------------------
```

This practical example demonstrates all three concepts. The variable `status` is local to the `display_info()` method. The attributes `name` and `score` belong to each student instance. The attribute `school` is defined at the class level and provides a shared school name. Together, these concepts help organize data and behavior within an object-oriented Python program.

## 3 Python Modifiers and Access Control — Part 1

Python uses naming conventions and special mechanisms to indicate how class attributes and methods are intended to be accessed. Unlike languages with strict access modifiers, Python mainly relies on naming conventions and name mangling to communi-

cate how class members should be used.

## 3.1 Public, Non-Public, and Private Naming Conventions

Python uses underscores in attribute and method names to indicate their intended accessibility.

- **Public:** Names without a leading underscore can be accessed from outside the class.
- **Non-public:** Names beginning with a single underscore indicate that an attribute or method is intended for internal use. This is a convention, not a strict restriction.
- **Private-style:** Names beginning with two leading underscores trigger name mangling, which helps prevent accidental name conflicts.

**Python Code:**

```python
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self._account_type = "Savings"
        self.__balance = balance

account = BankAccount("Dara", 1000)

print(account.owner)
print(account._account_type)
print(account._BankAccount__balance)
```

**Output:**

```
Dara
Savings
1000
```

In this example, `owner` is a public attribute that can be accessed directly. The attribute `_account_type` is intended for internal use by convention, but Python still permits direct access. The attribute `__balance` triggers name mangling, allowing it to be accessed using its mangled name. This mechanism helps prevent accidental access and naming conflicts, but it does not provide strict security.

## 3.2 Access Control by Convention and Name Mangling

Python communicates intended access through naming conventions rather than enforcing traditional private and protected access levels. A single leading underscore indicates internal use, while two leading underscores trigger name mangling. For example, `__balance` inside the `BankAccount` class becomes a name similar to `_BankAccount__balance`.

**Python Code:**

```python
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)

print("Balance:", account.get_balance())
print("Internal name:", account._BankAccount__balance)
```

**Output:**

```
Balance: 1000
Internal name: 1000
```

The `get_balance()` method provides a public way to retrieve the balance. The second statement demonstrates that name mangling does not make the attribute completely inaccessible. Instead, it mainly helps prevent accidental name conflicts, especially when working with inheritance.

### 3.3 The `@staticmethod` Decorator

A static method is defined using the `@staticmethod` decorator. It does not automatically receive an instance or class reference. Static methods are useful for utility operations related to a class that do not require instance or class attributes.

**Python Code:**

```python
class BankAccount:
    @staticmethod
    def is_valid_amount(amount):
        return amount > 0

print(BankAccount.is_valid_amount(500))
print(BankAccount.is_valid_amount(-100))
```

**Output:**

```
True
False
```

The `is_valid_amount()` method checks whether a transaction amount is positive. Because it does not need information from a particular account or the class itself, a static method is appropriate. It can be called directly through the class without creating an instance.

### 3.4 The `@classmethod` Decorator

A class method is defined using the `@classmethod` decorator. It receives the class itself as its first argument, conventionally named `cls`. It is useful for accessing or modifying class attributes shared by instances.

**Python Code:**

```python
class BankAccount:
    bank_name = "ITC Bank"

    @classmethod
    def change_bank_name(cls, new_name):
        cls.bank_name = new_name

BankAccount.change_bank_name("Cambodia Bank")

print(BankAccount.bank_name)
```

**Output:**

```
Cambodia Bank
```

The `change_bank_name()` method receives the class through `cls` and updates the class attribute `bank_name`. The updated value is available to instances that access the class attribute without defining their own attribute with the same name.

### 3.5 Practical Example: Bank Account Management

The following example combines public attributes, non-public attributes, a private-style balance, a static method for validating deposits, and a class method for changing the shared bank name.

**Python Code:**

```python
class BankAccount:
    bank_name = "ITC Bank"

    def __init__(self, owner, balance=0):
        self.owner = owner
        self._active = True
        self.__balance = balance

    def deposit(self, amount):
        if not self._active:
            print("Account is inactive.")
        elif self.is_valid_amount(amount):
            self.__balance += amount
            print("Deposit successful.")
        else:
            print("Invalid deposit amount.")

    def get_balance(self):
        return self.__balance

```

```
    @staticmethod
    def is_valid_amount(amount):
        return amount > 0

    @classmethod
    def change_bank_name(cls, new_name):
        cls.bank_name = new_name

account = BankAccount("Dara", 1000)

print("Owner:", account.owner)
print("Initial balance:", account.get_balance())

account.deposit(500)
print("Updated balance:", account.get_balance())

print("Valid amount:", BankAccount.is_valid_amount(200))

BankAccount.change_bank_name("Cambodia Bank")
print("Bank:", account.bank_name)
```

**Output:**

```
Owner: Dara
Initial balance: 1000
Deposit successful.
Updated balance: 1500
Valid amount: True
Bank: Cambodia Bank
```

This example demonstrates how Python organizes class data and behavior. The public attribute `owner` can be accessed directly, while `_active` indicates an internal-use attribute by convention. The private-style attribute `__balance` is accessed through the `get_balance()` method. The static method validates deposit amounts, while the class method updates the shared bank name. These techniques improve code organization, although a real banking application would require additional validation, authentication, and authorization safeguards.

# 4 Python Modifiers and OOP Features — Part 2

Python provides additional object-oriented programming features that help developers design reusable, maintainable, and reliable programs. These features include abstract classes, encapsulation, properties, and tools for indicating that classes or methods should not be overridden.

## 4.1 Abstract Classes and Methods

An abstract class is a class designed to serve as a blueprint for other classes. It cannot be instantiated directly when it contains abstract methods that have not been implemented. In Python, the `abc` module provides the `ABC` base class and the `@abstractmethod` decorator for defining abstract classes and methods.

An abstract method declares behavior that concrete subclasses must implement before their objects can be created.

**Python Code:**

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

circle = Circle(5)
print("Circle area:", round(circle.area(), 2))
```

**Output:**

```
Circle area: 78.54
```

In this example, `Shape` is an abstract class, and `area()` is an abstract method. The `Circle` subclass implements this method, allowing its objects to be created. Abstract classes help ensure that related subclasses provide the required behavior.

## 4.2 Access Control and Encapsulation

Encapsulation is an OOP principle that combines data and the methods that operate on it within a class. It also helps control how an object's internal state is accessed or modified. Python commonly uses naming conventions, methods, and properties to support encapsulation.

**Python Code:**

```python
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    @property
    def balance(self):
        return self.__balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            return True
        return False

account = BankAccount(1000)

```

```
print("Initial balance:", account.balance)
account.deposit(500)
print("Updated balance:", account.balance)
print("Deposit result:", account.deposit(-100))
```

**Output:**

```
Initial balance: 1000
Updated balance: 1500
Deposit result: False
```

The attribute `__balance` uses name mangling to reduce accidental access. The `balance` property provides a convenient way to read the value, while the `deposit()` method validates deposits before modifying the balance. This design encourages controlled access to the object's internal state, although Python does not provide strict private access enforcement.

## 4.3 Other OOP Features

Python provides additional tools for controlling attribute access and communicating design intentions. These include the `@property` decorator, `@final`, and `typing.final`.

### 4.3.1 The `@property` Decorator

The `@property` decorator allows a method to be accessed like an attribute. It is useful for calculating values or controlling access without requiring callers to use method-call syntax.

**Python Code:**

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    @property
    def area(self):
        return self.width * self.height

rectangle = Rectangle(5, 4)
print("Area:", rectangle.area)
```

**Output:**

```
Area: 20
```

The `area` property calculates the rectangle's area when accessed. It is used as `rectangle.area`, rather than `rectangle.area()`, making the calculated value appear like an ordinary attribute.

### 4.3.2 The `@final` Decorator and `typing.final`

The `final` decorator from the `typing` module indicates that a class or method is not intended to be subclassed or overridden. It communicates design intentions to type checkers and development tools. It does not enforce these restrictions at runtime.

**Python Code:**

```python
from typing import final

@final
class Payment:
    def process(self):
        return "Payment processed"

payment = Payment()
print(payment.process())
```

**Output:**

```
Payment processed
```

The `@final` decorator indicates that `Payment` should not be subclassed. Static type checkers can report violations of this intention, but Python still allows the class to be subclassed at runtime. The decorator therefore documents and checks design constraints rather than providing a runtime security mechanism.

## 4.4 Practical Example: Shape Area Calculator

The following example combines abstract classes, abstract methods, encapsulation, properties, and inheritance. The abstract `Shape` class defines a common interface, while `Circle` and `Rectangle` implement their own area calculations. The `area` property provides access to the calculated area without requiring a method call.

**Python Code:**

```python
from abc import ABC, abstractmethod
from math import pi
from typing import final

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

@final
class Circle(Shape):
    def __init__(self, radius):
        self.__radius = radius

    @property
    def radius(self):
        return self.__radius

```

```
    @property
    def area(self):
        return pi * self.__radius ** 2

class Rectangle(Shape):
    def __init__(self, width, height):
        self.__width = width
        self.__height = height

    @property
    def area(self):
        return self.__width * self.__height

circle = Circle(5)
rectangle = Rectangle(4, 6)

print("Circle radius:", circle.radius)
print("Circle area:", round(circle.area, 2))
print("Rectangle area:", rectangle.area)
```

**Output:**

```
Circle radius: 5
Circle area: 78.54
Rectangle area: 24
```

This example demonstrates several OOP features working together. The abstract `Shape` class requires subclasses to implement the `area()` method. The `Circle` and `Rectangle` classes provide their own area calculations through properties. Their dimensions use double-underscore names to reduce accidental access, while the `radius` property provides controlled read access to the circle's radius. Finally, `@final` indicates that `Circle` is not intended to be subclassed, although this restriction is checked by static type checkers rather than enforced by Python at runtime.

# 5 Python Operators — Part 1

Operators are symbols or keywords used to perform operations on values and variables. Python provides several categories of operators, including arithmetic, comparison, bitwise, and logical operators. These operators are essential for performing calculations, comparing values, manipulating binary data, and making decisions in a program.

## 5.1 Arithmetic Operators

Arithmetic operators perform mathematical calculations on numeric values. Python provides addition, subtraction, multiplication, division, floor division, modulus, and exponentiation.

- **Addition (+):** Adds two values.
- **Subtraction (-):** Subtracts one value from another.
- **Multiplication (*):** Multiplies two values.

- **Division (/):** Divides one value by another and returns a floating-point result.
- **Floor division (//):** Divides two values and rounds the quotient down to the nearest integer value.
- **Modulus (%):** Returns the remainder after division.
- **Exponentiation (**):** Raises a value to a power.

**Python Code:**

```
a = 10
b = 3

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor division:", a // b)
print("Modulus:", a % b)
print("Exponentiation:", a ** b)
```

**Output:**

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Floor division: 3
Modulus: 1
Exponentiation: 1000
```

In this example, the operators perform different calculations using the same two values. Division returns a floating-point result, floor division rounds the quotient down, and modulus returns the remainder. Exponentiation calculates the first value raised to the power of the second value.

## 5.2 Comparison Operators

Comparison operators compare two values and return a Boolean result: `True` or `False`. They are commonly used in conditional statements and eligibility checks.

- **Equal to (==):** Checks whether two values are equal.
- **Not equal to (!=):** Checks whether two values are different.
- **Greater than (>):** Checks whether the left value is greater than the right value.
- **Less than (<):** Checks whether the left value is less than the right value.
- **Greater than or equal to (>=):** Checks whether the left value is greater than or equal to the right value.
- **Less than or equal to (<=):** Checks whether the left value is less than or equal to the right value.

**Python Code:**

```
score = 75
passing_score = 50

print("Equal:", score == passing_score)
print("Not equal:", score != passing_score)
print("Greater than:", score > passing_score)
print("Less than:", score < passing_score)
print("At least 50:", score >= passing_score)
print("At most 50:", score <= passing_score)
```

**Output:**

```
Equal: False
Not equal: True
Greater than: True
Less than: False
At least 50: True
At most 50: False
```

The comparison operators evaluate the relationship between the student's score and the passing score. For example, `score >= passing_score` returns `True` because 75 is greater than 50.

### 5.3 Bitwise Operators

Bitwise operators perform operations on the binary representations of integers. They are commonly used in low-level programming, bit flags, and situations where individual bits must be manipulated.

- **AND (&):** Sets a result bit to 1 when both corresponding bits are 1.
- **OR (|):** Sets a result bit to 1 when at least one corresponding bit is 1.
- **XOR (^):** Sets a result bit to 1 when the corresponding bits are different.
- **NOT (~):** Inverts the bits of an integer according to Python's signed integer representation.
- **Left shift (<<):** Shifts bits to the left.
- **Right shift (>>):** Shifts bits to the right.

**Python Code:**

```
a = 6  # Binary: 110
b = 3  # Binary: 011

print("AND:", a & b)
print("OR:", a | b)
print("XOR:", a ^ b)
print("NOT:", ~a)
print("Left shift:", a << 1)
print("Right shift:", a >> 1)
```

**Output:**

```
AND: 2
OR: 7
XOR: 5
NOT: -7
Left shift: 12
Right shift: 3
```

The AND, OR, and XOR operators compare corresponding binary bits. For example, the binary representations of 6 and 3 are `110` and `011`. Their AND operation produces `010`, which represents 2. In Python, `~6` evaluates to -7 because integer bitwise NOT follows the relationship `~x = -x - 1`.

### 5.4 Logical Operators

Logical operators combine or reverse Boolean conditions. They are commonly used to control program flow and evaluate whether multiple requirements are satisfied.

- **AND (`and`):** Returns `True` when both conditions are true.
- **OR (`or`):** Returns `True` when at least one condition is true.
- **NOT (`not`):** Reverses the truth value of a condition.

**Python Code:**

```
score = 85
attendance = 90

print("Passed:", score >= 50 and attendance >= 75)
print("Eligible for award:", score >= 90 or attendance >= 95)
print("Failed:", not (score >= 50))
```

**Output:**

```
Passed: True
Eligible for award: False
Failed: False
```

The `and` operator requires both conditions to be true, while `or` requires at least one condition to be true. The `not` operator reverses a Boolean result. These operators make it possible to evaluate multiple requirements in a single expression.

### 5.5 Practical Example: Student Grades and Eligibility

A student evaluation program can use arithmetic operators to calculate average scores, comparison operators to determine academic performance, and logical operators to check eligibility. The following example evaluates a student's average score and attendance to determine whether the student passes and qualifies for an academic award.

**Python Code:**

```
student_name = "Dara"
math_score = 85
programming_score = 92
attendance = 90

average = (math_score + programming_score) / 2
passed = average >= 50 and attendance >= 75
eligible_for_award = average >= 90 or (
    average >= 85 and attendance >= 90
)

print("Student:", student_name)
print("Average score:", average)
print("Passed:", passed)
print("Eligible for award:", eligible_for_award)
```

**Output:**

```
Student: Dara
Average score: 88.5
Passed: True
Eligible for award: True
```

This example demonstrates how different operators work together to solve a practical problem. Arithmetic operators calculate the student's average score. Comparison operators check the average and attendance against required thresholds. Logical operators combine these conditions to determine whether the student passes and qualifies for an academic award. This approach makes program decisions clearer and easier to maintain.

# 6 Python Operators — Part 2

Python provides additional operators for updating variables, checking object identity, testing membership in collections, and selecting values conditionally. Understanding operator precedence and associativity is also important because these rules determine how Python evaluates complex expressions.

## 6.1 Assignment Operators

Assignment operators assign values to variables or update existing values. The basic assignment operator (=) stores a value in a variable, while augmented assignment operators combine an operation with assignment.

Common assignment operators include:

- **Assignment (=):** Assigns a value to a variable.
- **Addition assignment (+=):** Adds a value and assigns the result.
- **Subtraction assignment (-=):** Subtracts a value and assigns the result.

- **Multiplication assignment (*=):** Multiplies a value and assigns the result.
- **Division assignment (/=):** Divides a value and assigns the result.
- **Floor division assignment (//=):** Performs floor division and assigns the result.
- **Modulus assignment (%=):** Assigns the remainder after division.
- **Exponentiation assignment (**=)**: Raises a value to a power and assigns the result.

**Python Code:**

```
score = 80
score += 5
print("After addition:", score)

score -= 10
print("After subtraction:", score)

score *= 2
print("After multiplication:", score)

score /= 2
print("After division:", score)

score //= 3
print("After floor division:", score)

score %= 10
print("After modulus:", score)

score **= 2
print("After exponentiation:", score)
```

**Output:**

```
After addition: 85
After subtraction: 75
After multiplication: 150
After division: 75.0
After floor division: 25.0
After modulus: 5.0
After exponentiation: 25.0
```

This example demonstrates how assignment operators update the same variable step by step. Each operation uses the value produced by the previous operation. Assignment operators make code shorter and more readable when a variable needs to be updated repeatedly.

## 6.2 Special Operators

Special operators include identity operators, membership operators, and conditional expressions. They help determine whether two references point to the same object, whether a value exists in a collection, and which value should be selected based on a condition.

### 6.2.1 Identity Operators

Identity operators compare the identity of two objects rather than simply comparing their values.

- **Identity (`is`):** Returns `True` if two references point to the same object.
- **Not identical (`is not`):** Returns `True` if two references point to different objects.

**Python Code:**

```
first_list = [80, 90, 85]
second_list = first_list
third_list = [80, 90, 85]

print(first_list is second_list)
print(first_list is third_list)
print(first_list is not third_list)
print(first_list == third_list)
```

**Output:**

```
True
False
True
True
```

The variables `first_list` and `second_list` refer to the same list object, so the identity comparison returns `True`. The variable `third_list` refers to a different list object with the same contents. Therefore, `is` returns `False`, while `==` returns `True` because the lists have equal values.

### 6.2.2 Membership Operators

Membership operators check whether a value exists in a collection, such as a list, tuple, set, or dictionary.

- **Membership (`in`):** Returns `True` if a value is found in a collection.
- **Non-membership (`not in`):** Returns `True` if a value is not found in a collection.

**Python Code:**

```
subjects = ["Python", "Mathematics", "Database"]

print("Python" in subjects)
print("Physics" in subjects)
print("Physics" not in subjects)
```

**Output:**

```
True
False
True
```

The `in` operator checks whether a subject appears in the list, while `not in` checks whether it is absent. For dictionaries, membership tests keys by default rather than values.

### 6.2.3 Conditional Expressions

A conditional expression selects one of two values based on a condition. Its syntax is `value_if_true if condition else value_if_false`.

**Python Code:**

```
score = 72

result = "Passed" if score >= 50 else "Failed"

print("Score:", score)
print("Result:", result)
```

**Output:**

```
Score: 72
Result: Passed
```

The conditional expression checks whether the score is at least 50. Because the condition is true, the expression assigns `"Passed"` to `result`. This form is useful for simple decisions that can be expressed clearly in one line.

## 6.3 Operator Precedence and Associativity

Operator precedence determines which operators are evaluated first in an expression. Associativity determines the evaluation order when operators have the same precedence. Parentheses can be used to make the intended order explicit.

For example, multiplication has higher precedence than addition, so multiplication is evaluated first. Exponentiation is evaluated from right to left, while most arithmetic operations of the same precedence are evaluated from left to right.

**Python Code:**

```python
result1 = 2 + 3 * 4
result2 = (2 + 3) * 4
result3 = 2 ** 3 ** 2
result4 = 20 / 5 * 2

print("Multiplication first:", result1)
print("Parentheses first:", result2)
print("Exponentiation:", result3)
print("Left to right:", result4)
```

**Output:**

```
Multiplication first: 14
Parentheses first: 20
Exponentiation: 512
Left to right: 8.0
```

In the first expression, multiplication is performed before addition, producing 14. Parentheses change the order in the second expression, producing 20. Exponentiation is right-associative, so `2 ** 3 ** 2` is evaluated as `2 ** (3 ** 2)`, producing 512. Division and multiplication have the same precedence and are evaluated from left to right in the final expression.

## 6.4 Practical Example: Student Record Management

The following example combines assignment operators, identity and membership operators, conditional expressions, and operator precedence to update a student's score, check subject enrollment, and determine the student's result.

**Python Code:**

```python
class Student:
    def __init__(self, name, score, subjects):
        self.name = name
        self.score = score
        self.subjects = subjects

student = Student(
    "Dara", 80, ["Python", "Mathematics"]
)

same_student = student

student.score += 5
student.subjects += ["Database"]

has_python = "Python" in student.subjects
has_physics = "Physics" in student.subjects
passed = student.score >= 50
result = "Passed" if passed else "Failed"
final_score = (student.score + 10) / 2

print("Student:", student.name)
```

```
print("Score:", student.score)
print("Final score:", final_score)
print("Subjects:", student.subjects)
print("Enrolled in Python:", has_python)
print("Not enrolled in Physics:", "Physics" not in student.subjects)
print("Same object:", student is same_student)
print("Result:", result)
```

**Output:**

```
Student: Dara
Score: 85
Final score: 47.5
Subjects: ['Python', 'Mathematics', 'Database']
Enrolled in Python: True
Not enrolled in Physics: True
Same object: True
Result: Passed
```

This example demonstrates how different operators can work together in a student management program. The `+=` operator updates the student's score and subject list. The membership operators check whether particular subjects are present, while the identity operator verifies that two variables refer to the same student object. The conditional expression determines the student's result, and arithmetic operators calculate a separate final-score value. Parentheses make the intended calculation order explicit.

# 7 Python Decision-Making Statements — Part 1

Decision-making statements allow a Python program to execute different blocks of code depending on whether certain conditions are true or false. They are useful for making decisions based on student scores, attendance, eligibility, and other conditions.

## 7.1 The `if` Statement

The `if` statement executes a block of code only when its condition is true. If the condition is false, Python skips that block and continues with the next statement. The code inside the `if` block must be indented.

**Python Code:**

```
score = 85

if score >= 50:
    print("The student passed the exam.")

print("Result checking completed.")
```

**Output:**

```
The student passed the exam.
Result checking completed.
```

In this example, the condition `score >= 50` is true because the student's score is 85. Therefore, Python executes the indented `print()` statement.

### 7.2 The `if-else` Statement

The `if-else` statement provides two possible paths. Python executes the `if` block when the condition is true; otherwise, it executes the `else` block. This is useful when a program must choose between two outcomes.

**Python Code:**

```
score = 42

if score >= 50:
    print("Pass")
else:
    print("Fail")
```

**Output:**

```
Fail
```

Since the student's score is below 50, the condition is false, so Python executes the `else` block and displays `Fail`.

### 7.3 The `if-elif-else` Statement

The `if-elif-else` statement is used when a program needs to evaluate multiple conditions. Python checks the conditions from top to bottom and executes the first block whose condition is true. If none of the conditions is true, the `else` block executes.

**Python Code:**

```
score = 87

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
elif score >= 50:
    print("Grade: E")
else:
    print("Grade: F")
```

**Output:**

```
Grade: B
```

The score of 87 does not satisfy the first condition because it is below 90. However, it satisfies `score >= 80`, so Python assigns Grade B and skips the remaining conditions.

### 7.4 Practical Example: Student Pass/Fail Status and Letter Grades

Decision-making statements can be combined to evaluate student performance. The following example determines whether a student passes or fails and assigns a letter grade based on the student's score.

For this example, a score of 50 or higher is considered a pass. The grading scale assigns A to scores of 90 or above, B to 80–89, C to 70–79, D to 60–69, E to 50–59, and F to scores below 50.

**Python Code:**

```
student_name = "Dara"
score = 87

if score >= 50:
    status = "Pass"
else:
    status = "Fail"

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
elif score >= 50:
    grade = "E"
else:
    grade = "F"

print("Student:", student_name)
print("Score:", score)
print("Status:", status)
print("Letter Grade:", grade)
```

**Output:**

```
Student: Dara
Score: 87
Status: Pass
Letter Grade: B
```

In this practical example, the program first determines the student's pass/fail status using an `if-else` statement. It then uses an `if-elif-else` statement to assign the appropriate letter grade. Since Dara scores 87, the program identifies the result as Pass and assigns Grade B.

This example demonstrates how decision-making statements help programs evaluate conditions and produce appropriate results automatically.

# 8 Python Decision-Making Statements — Part 2

Python provides additional ways to control program execution when a problem requires multiple levels of conditions, several menu options, or a short expression that chooses between two values. These techniques make programs more flexible and easier to organize.

## 8.1 Nested `if` Statements

A nested `if` statement is an `if` statement placed inside another `if` or `else` block. It allows a program to check a second condition only after the first condition has been satisfied.

**Python Code:**

```
score = 85
attendance = 90

if score >= 50:
    if attendance >= 80:
        print("Student passed.")
        print("Student meets the attendance requirement.")
    else:
        print("Student passed but attendance is insufficient.")
else:
    print("Student failed.")
```

**Output:**

```
Student passed.
Student meets the attendance requirement.
```

In this example, the program first checks whether the score is at least 50. If the student passes, it checks whether attendance is at least 80. Both conditions are true, so the program displays both messages.

## 8.2 Multiple-Branch Selection Using `match-case`

The `match-case` statement, introduced in Python 3.10, allows a program to select an action based on the value of an expression. Each `case` represents a possible pattern. The underscore (_) acts as a wildcard and matches any value not handled by the preceding cases.

**Python Code:**

```
choice = 2

match choice:
    case 1:
        print("Add a student")
    case 2:
        print("View students")
    case 3:
        print("Search for a student")
    case 4:
        print("Exit the program")
    case _:
        print("Invalid menu option")
```

**Output:**

```
View students
```

Because the value of `choice` is 2, Python executes the corresponding `case 2` block. The other cases are skipped.

The same menu selection can also be implemented using `if-elif-else`, which is useful when working with Python versions older than 3.10.

**Python Code:**

```
choice = 2

if choice == 1:
    print("Add a student")
elif choice == 2:
    print("View students")
elif choice == 3:
    print("Search for a student")
elif choice == 4:
    print("Exit the program")
else:
    print("Invalid menu option")
```

**Output:**

```
View students
```

Both approaches produce the same result in this example. The `match-case` statement is convenient for matching specific values, while `if-elif-else` is more flexible when conditions involve comparisons or logical expressions.

## 8.3 Conditional Expressions

A conditional expression provides a shorter way to choose between two values based on a condition. In Java, a similar operation uses the ternary operator, written as `condition`

`?` `value1` `:` `value2`. In Python, the syntax is `value1 if condition else value2`.

**Python Code:**

```
score = 76

status = "Pass" if score >= 50 else "Fail"
grade = "Good" if score >= 70 else "Needs improvement"

print("Status:", status)
print("Performance:", grade)
```

**Output:**

```
Status: Pass
Performance: Good
```

The first expression assigns `Pass` because the score is at least 50. The second expression assigns `Good` because the score is at least 70. Conditional expressions are most suitable for simple decisions involving two possible values.

## 8.4 Practical Example: Menu-Driven Student Management System

A menu-driven program allows users to choose actions from a list of options. The following example combines nested `if` statements, `match-case`, and conditional expressions to manage basic student records. The menu supports adding a student, viewing records, searching by name, and exiting the program.

**Python Code:**

```
students = []

while True:
    print("\n--- Student Management System ---")
    print("1. Add student")
    print("2. View students")
    print("3. Search student")
    print("4. Exit")

    choice = input("Enter your choice: ")

    match choice:
        case "1":
            name = input("Enter student name: ").strip()

            if name:
                score = float(input("Enter student score: "))

                if 0 <= score <= 100:
                    students.append({
                        "name": name,
                        "score": score
                    })
                    print("Student added successfully.")
                else:
```

```
                    print("Score must be between 0 and 100.")
            else:
                print("Student name cannot be empty.")

        case "2":
            if students:
                for student in students:
                    status = (
                        "Pass" if student["score"] >= 50 else "Fail"
                    )
                    print(
                        f'{student["name"]}: '
                        f'{student["score"]} - {status}'
                    )
            else:
                print("No student records found.")

        case "3":
            search_name = input("Enter name to search: ").strip()
            found = False

            for student in students:
                if student["name"].lower() == search_name.lower():
                    status = (
                        "Pass" if student["score"] >= 50 else "Fail"
                    )
                    print(
                        f'{student["name"]}: '
                        f'{student["score"]} - {status}'
                    )
                    found = True
                    break

            if not found:
                print("Student not found.")

        case "4":
            print("Exiting the program.")
            break

        case _:
            print("Invalid choice. Please try again.")
```

**Example Output:**

```
--- Student Management System ---
1. Add student
2. View students
3. Search student
4. Exit
Enter your choice: 1
Enter student name: Dara
Enter student score: 85
Student added successfully.

--- Student Management System ---
1. Add student
2. View students
3. Search student
4. Exit
Enter your choice: 2
Dara: 85.0 - Pass

--- Student Management System ---
1. Add student
2. View students
3. Search student
4. Exit
Enter your choice: 3
Enter name to search: Dara
Dara: 85.0 - Pass

--- Student Management System ---
1. Add student
2. View students
3. Search student
4. Exit
Enter your choice: 4
Exiting the program.
```

This practical example demonstrates how different decision-making techniques work together in a real application. The `match-case` statement processes menu choices, nested `if` statements validate student information, and conditional expressions determine pass/fail status. The `while` loop keeps the menu running until the user selects the exit option.

The program stores records in a list while it is running. The records are not saved permanently, so they will be lost when the program terminates.

# 9 Python Loops and Loop Control

Loops are control structures that allow a program to execute a block of code repeatedly. They help reduce code duplication and are useful for processing collections of data, calculating student averages, and searching for specific records. Python provides `while` and `for` loops, along with loop control statements that change how repetition works.

## 9.1 The `while` Loop

The `while` loop repeatedly executes a block of code as long as its condition remains true. It is useful when the number of repetitions is not known in advance. The condition must eventually become false to prevent an infinite loop.

**Python Code:**

```
count = 1

while count <= 5:
    print("Count:", count)
    count += 1
```

**Output:**

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```

In this example, the loop begins with `count = 1`. After each iteration, the value increases by one. When `count` becomes 6, the condition is false, and the loop terminates.

## 9.2 The `for` Loop

The `for` loop iterates over the items in a sequence or another iterable object, such as a list, tuple, string, or range. It is commonly used when processing a collection or repeating an operation a specific number of times.

**Python Code:**

```
subjects = ["Python", "Mathematics", "Database"]

for subject in subjects:
    print(subject)
```

**Output:**

```
Python
Mathematics
Database
```

The loop assigns each item in the `subjects` list to the variable `subject` and prints it. The loop ends automatically after all items have been processed.

## 9.3 Repetition with `for` and `while`

Both `for` and `while` loops support repetition, but they are commonly used in different situations. A `for` loop is convenient when iterating through a known sequence or range,

while a `while` loop is useful when repetition depends on a condition.

Python does not have a native `do-while` loop like Java. However, similar behavior can be achieved using `while True` and `break`, ensuring that the loop body executes at least once.

**Python Code:**

```python
# Using a for loop
for number in range(1, 4):
    print("For loop:", number)

# Using a while loop
number = 1
while number <= 3:
    print("While loop:", number)
    number += 1

# Simulating a do-while loop
number = 1
while True:
    print("Do-while style:", number)
    number += 1

    if number > 3:
        break
```

**Output:**

```
For loop: 1
For loop: 2
For loop: 3
While loop: 1
While loop: 2
While loop: 3
Do-while style: 1
Do-while style: 2
Do-while style: 3
```

All three loops repeat three times. The final loop simulates do-while behavior because its body executes before the stopping condition is checked.

## 9.4 Loop Control Statements: `break`, `continue`, and `pass`

Python provides three useful statements for controlling loop behavior:

- `break`: Immediately terminates the nearest enclosing loop.
- `continue`: Skips the remaining statements in the current iteration and proceeds to the next iteration.
- `pass`: Performs no operation and serves as a placeholder when a statement is syntactically required.

**Python Code:**

```python
# Using continue
for number in range(1, 6):
    if number == 3:
        continue
    print(number)

# Using pass
for number in range(1, 4):
    if number == 2:
        pass
    print("Number:", number)
```

**Output:**

```
1
2
4
5
Number: 1
Number: 2
Number: 3
```

The `continue` statement skips printing the number 3 in the first loop. In the second loop, `pass` does nothing, so the program continues to execute the following `print()` statement normally.

### 9.5 The `break` Statement

The `break` statement immediately exits a loop when a specified condition is met. It is useful when a program finds the required information or when further iteration is unnecessary.

**Python Code:**

```python
scores = [75, 82, 91, 68, 88]

for score in scores:
    if score >= 90:
        print("High score found:", score)
        break
    print("Checking score:", score)
```

**Output:**

```
Checking score: 75
Checking score: 82
High score found: 91
```

When the loop encounters the score 91, the condition `score >= 90` becomes true. Python prints the message and executes `break`, ending the loop without checking the remaining scores.

## 9.6 Practical Example: Student Objects, Averages, and Searching

Loops can process objects stored in a collection. In this example, a `Student` class stores each student's name and scores. The program iterates through student objects, calculates their average scores, determines their pass/fail status, and searches for a student by name.

**Python Code:**

```python
class Student:
    def __init__(self, name, scores):
        self.name = name
        self.scores = scores

    def calculate_average(self):
        return sum(self.scores) / len(self.scores)


students = [
    Student("Dara", [80, 85, 90]),
    Student("Sokha", [45, 50, 55]),
    Student("Mony", [92, 88, 95])
]

total_average = 0

print("--- Student Results ---")

for student in students:
    average = student.calculate_average()
    total_average += average

    status = "Pass" if average >= 50 else "Fail"

    print(f"Name: {student.name}")
    print(f"Average: {average:.2f}")
    print(f"Status: {status}")

class_average = total_average / len(students)
print(f"\nClass Average: {class_average:.2f}")

search_name = "Sokha"
found = False

for student in students:
    if student.name.lower() == search_name.lower():
        print(f"\nFound student: {student.name}")
        print(f"Average: {student.calculate_average():.2f}")
        found = True
        break

if not found:
    print("\nStudent not found.")
```

**Output:**

```
--- Student Results ---
Name: Dara
Average: 85.00
```

```
Status: Pass
Name: Sokha
Average: 50.00
Status: Pass
Name: Mony
Average: 91.67
Status: Pass

Class Average: 75.56

Found student: Sokha
Average: 50.00
```

In this example, the `for` loop processes each student object and calculates the average of the student's scores. The program also calculates the overall class average and determines whether each student passes. A second loop searches for Sokha by name. Once the student is found, `break` terminates the search loop, avoiding unnecessary iterations.

## 9.7 Important Differences Between Java and Python

Although Java and Python both support loops and loop control statements, they differ in syntax and behavior.

Table 1: Comparison of Java and Python Loop Structures

| **Feature** | **Java** | **Python** |
|---|---|---|
| Block Structure | Uses curly braces {} to define loop bodies. | Uses indentation to define loop bodies. |
| `for` Loop | Commonly uses initialization, condition, and update expressions. | Iterates over items in an iterable or uses `range()`. |
| `while` Loop | Repeats while its condition is true. | Repeats while its condition is true. |
| Do-while Loop | Supports a native `do-while` loop that executes at least once. | Has no native `do-while` loop; similar behavior can be implemented using `while True` and `break`. |
| Loop Control | Supports `break` and `continue`. | Supports `break` and `continue`. |
| Empty Block | Can use an empty block written as {}. | Uses `pass` when a block requires a statement but no action is needed. |
| Range Iteration | Often uses a counter, condition, and update expression. | Can use `range(1, 4)` to generate 1, 2, and 3. |
| Syntax | Statements generally end with semicolons, and blocks use braces. | Uses colons and indentation; semicolons are usually unnecessary. |

For example, a Java loop that prints numbers from 1 to 3 can be written as:

**Java Code:**

```java
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            System.out.println(i);
        }
    }
}
```

**Output:**

```
1
2
3
```

The equivalent Python code is:

**Python Code:**

```python
for i in range(1, 4):
    print(i)
```

**Output:**

```
1
2
3
```

Java uses braces and an explicit counter update in this example, whereas Python uses indentation and the `range()` function. Understanding these differences helps programmers transfer their knowledge between the two languages.

## 10 Conclusion

This report introduced the fundamental concepts of Python programming, including data types, variables, attributes, modifiers, operators, decision-making statements, and loops. Through practical student management examples, it demonstrated how these concepts can be applied to solve real-world problems. The comparison between Java and Python also highlighted their key differences. These fundamentals provide a strong foundation for further study of object-oriented programming and software development.

## 11 References

1. Python Software Foundation. (n.d.). *The Python Tutorial*. https://docs.python.org/3/tutorial/
2. Python Software Foundation. (n.d.). *More Control Flow Tools*. https://docs.python.org/3/tutorial/controlflow.html
3. Python Software Foundation. (n.d.). *Classes*. https://docs.python.org/3/tutorial/classes.html
4. Oracle. (n.d.). *The Java Tutorials: Language Basics*. https://docs.oracle.com/javase/tutorial/java/nutsandbolts/